

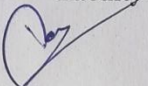
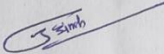
MITRC
MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of faculty: Dazy Arya
Subject & Subject Code: CF&P, 2FY2-08
Semester: II
Session: 2022-23 (Even Sem.)
Branch: AI&DS, ME, CE, EE
Batch: C, D
UNIT: III
Date of Submission: 28/03/2023

Lecture No.	Topics Covered	Total Page
1	Control statement and loop in C-if statement, nested if, if-else,,	4
2	else-if, switch statement,	4
3	, goto statement, conditional (? :) statement	4
4	Looping iteration-basic iteration(initialization, steps, termination), while statement, do while,	3
5	, For statement, break statement, continue statement.	5

References:

1. Brian W. Kernighan and Dennis Ritchie, C Programming Language, Prentice Hall of India.
2. Yashwant Kanetkar, Let US C, BPB Publication.

Faculty:  HOD:  Dean Academics:  28/3/2023

REDMI NOTE 10 LITE
AI QUAD CAMERA

LECTURE -01 (UNIT-03)**Control statements in C**

Control statements control the flow of execution of the statements of a program. The various types of control statements in C language are as under:-

- I. Sequential
- II. Conditional
- III. Iteration

Sequential control:- In sequential control, the C program statements are executed sequentially i.e., one after the another from beginning to end.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
int x , y, sum;
```

```
printf("enter the two numbers");
```

```
scanf("%d%d",&x,&y);
```

```
sum=x+y;
```

```
printf("the sum of the two numbers is =%d",sum);
```

```
getch();
```

```
}
```

Conditional Control (Selection Control or Decision Control) :- In conditional control , the execution of

statements depends upon the condition-test. If the condition evaluates to true, then a set of statements is

executed otherwise another set of statements is followed. This control is also called Decision Control

because it helps in making decision about which set of statements is to be executed.

Decision control structure in C can be implemented by using:-

1. If statement
2. If-else statement
3. Nested if else statement
4. else-if ladder
5. case control structure
6. conditional operator

Statement1

Statement2

Statement3

Iteration Control (Loops)

Iterations or loops are used when we want to execute a statement or block of statements several times.

The repetition of loops is controlled with the help of a test condition. The statements in the loop keep on

executing repetitively until the test condition becomes false.

There are the following three types of loops:-

1. While loop
2. Do-while loop
3. For loop

Decision control

Conditional Control (Selection Control or Decision Control) :- In conditional control , the execution of

statements depends upon the condition-test. If the condition evaluates to true, then a set of statements is

executed otherwise another set of statements is followed. This control is also called Decision Control

because it helps in making decision about which set of statements is to be executed.

Decision control structure in C can be implemented by using:-

1. If statement
2. If-else statement
3. Nested if else statement
4. else-if ladder
5. case control structure
6. conditional operator

if statement:- This is the most simple form of decision control statement. In this form, a set of statements are executed only if the condition given with if evaluates to true.

Its general syntax and flow chart is as under:-

```
if(condition)
{
    Statements ;
}
```

A program to understand the if statement.

/* program to check whether the given number is negative*

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int num;
    clrscr();
    printf("enter the number");
    scanf("%d",&num);
```

```
if(num<0)
{
printf("the entered number is negative");
}
getch();
}
```

if- else statement:- This is a bi-directional control statement. This statement is used to test a condition and take one of the two possible actions. If the condition evaluates to true then one statement (or block of statements) is executed otherwise other statement (or block of statements) is executed. The general form and flow chart of if-else statement is as under:-

```
if(condition)
{
block of statements;
}
else
{
block of statements;
}
```

We illustrate if-else statement using the following program

```
/*Program to find the biggest of two numbers*/
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int num1,num2 ;
```

```
clrscr();
```

```
printf("enter the two numbers");
```

```
scanf("%d %d",&num1,&num2);
```

```
if(num1>num2)
```

```
{
```

```
printf("the number %d is big",num1);
```

```
}
```

```
else
```

```
{
```

```
printf("the number %d is big",num2);
```

```
}
```

```
getch();
```

```
}
```

References:

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.



LECTURE -02 (UNIT-03)

Nested if-else:- If we have if-else statement within either the body of an if statement or the body of else statement or in the body of both if and else, then this is known as nesting of if else statement.

The general form of nested if-else statement is as follows:-

```
if(condition1)
```

```
{
```

```
if(condition2)
```

```
statements;
```

```
else
```

```
statements;
```

```
}
```

```
else
```

```
{
```

```
if(condition3)
```

```
Statements;
```

```
else
```

```
Statements;
```

```
}
```

We give the following program to explain nesting of if else:-

```
/*program to find the largest of three numbers*/
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int a, b,c,large;
```

```
printf("enter the three numbers");
```

```
scanf("%d%d%d",&a,&b,&c);
```

```
if(a>b)
```

```
{
```

```
if(a>c)
```

```
large=a;
```

```
else
```

```
large=c;
```

```
}
```

```
else
```

```
{
```

```
if(b>c)
```

```
large=b;
```

```
else
```

```
large=c;
```

```
}
```

```
printf("largest number is %d",large);
getch();
}
```

Else if ladder:- This is a type of nesting in which there is an if-else statement in every else part except the last else part. This type of nesting is called else if ladder.

The general syntax and flow chart of else if ladder is as follows:-

```
If(condition1)
statementA;
else if(condition2)
statementB;
else if(condition3)
statementC;
else
statementD;
```

A program to illustrate the else if ladder:-

```
/*program to find the grade of the student*/
#include<stdio.h>
#include<conio.h>
void main()
{
int marks;
printf("enter the percentage of the student");
scanf("%d",&marks);
if(marks>=80)
printf("your grade is a");
else if(marks>=70)
printf("your grade is b");
else if(marks>=60)
printf("your grade is c");
else if(marks>=50)
printf("your grade is d");
else
printf("you are fail");
getch();
}
```

Switch case statement :- Switch case statements are a substitute for long if statements and has more flexibility and a clearer format than else-if ladder.

This statement is used to select one out of the several numbers of alternatives present in a block. This selection statement successively tests the value of an expression against a list of integer or character constants. When a match is found, the statements associated with that constant are executed.

The general syntax of switch case statement is as follows:-

```
switch(expression)
{
case constant1: statements;
case constant2: statements;
case constant3: statements;
.....
.....
case constantN: statements;
default : statement;
}
```

Here switch, case and default are keywords.

The expression following the switch keyword must give an integer value. This expression can be any

integer or character variable, or a function call that returns an integer. It can also be arithmetic, relational,

logical or bitwise expression yielding an integer. It can be integer or character constant also.

Data types

long int and short int are also allowed.

The constant following the case keyword should be of integer or character type. We cannot use floating

point or string constant.

A program to explain switch case statement

```
/*program to print day of the week*/
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int day;
```

```
clrscr();
```

```
printf("enter the day number from 1 to7");
```

```
scanf("%d",&day);
```

```
switch(day)
```

```
{
```

```
case 1: printf("monday");
```

```
break;
```

```
case 2:printf("tuesday");
```

```
break;
```

```
case 3: printf("wednesday");
```

```
break;
```

```
case 4:printf("thursday");
```

```
break;
```

```
case 5: printf("friday");
```

```
break;
```

```
case 6:printf("saturday");
```



```
break;
case 7: printf("sunday");
break;
default:printf("wrong input");
}
getch();
}
```

NOTE:- In switch case statement, if we do not associate a break statement with every case then from the case for which the expression matches with the constant, all the statements are executed until the switch case block ends. This situation is referred to as Fall Through.

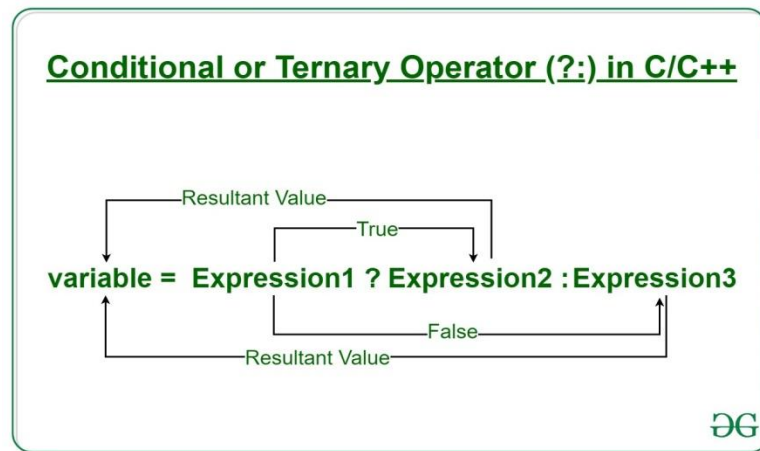
References:

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.



LECTURE -03 (UNIT-03)

Conditional operator:- The conditional operator is kind of similar to the if-else statement as it does follow the same algorithm as of if-else statement but the conditional operator takes less space and helps to write the if-else statements in the shortest way possible.

**Syntax:**

The conditional operator is of the form

variable = Expression1 ? Expression2 : Expression3

Or the syntax will also be in this form

variable = (condition) ? Expression2 : Expression3

Or syntax will also be in this form

(condition) ? (variable = Expression2) : (variable = Expression3)

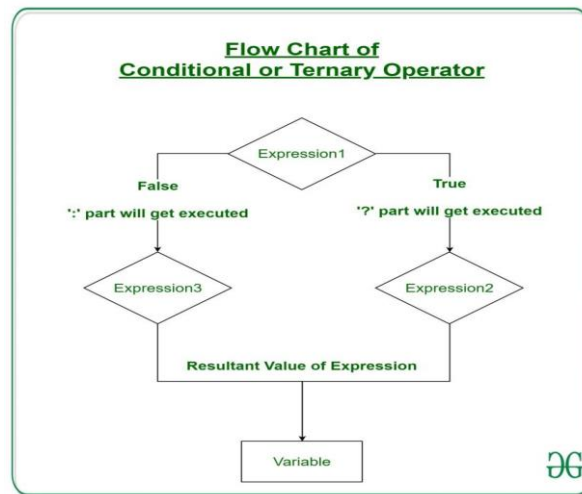
It can be visualized into if-else statement as:

```
if(Expression1)
{
    variable = Expression2;
}
else
{
    variable = Expression3;
}
```

Since the Conditional Operator ‘?:’ takes three operands to work, hence they are also called **ternary operators**.

Working:

Here, **Expression1** is the condition to be evaluated. If the condition(**Expression1**) is True then **Expression2** will be executed and the result will be returned. Otherwise, if the condition(**Expression1**) is false then **Expression3** will be executed and the result will be returned.



Example 1: Program to Store the greatest of the two Number.

// C program to find largest among two

// numbers using ternary operator

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int m = 5, n = 4;
```

```
    (m > n) ? printf("m is greater than n that is %d > %d",
```

```
        m, n)
```

```
        : printf("n is greater than m that is %d > %d",
                n, m);

    return 0;
}
```

Output

m is greater than n that is 5 > 4

Example 2: Program to check whether a year is leap year or not.

// C program to check whether a year is leap year or not

// using ternary operator

```
#include <stdio.h>

int main()
{
    int yr = 1900;

    (yr%4==0) ? (yr%100!=0? printf("The year %d is a leap year",yr)
    : (yr%400==0 ? printf("The year %d is a leap year",yr)
    : printf("The year %d is not a leap year",yr)))
    : printf("The year %d is not a leap year",yr);

    return 0;
}
```

//This code is contributed by Susobhan AKhuli

Output

The year 1900 is not a leap year

goto statement:- It is an unconditional statement and it transfers the control to the another part of the

program. The goto statement is used as under:-

goto label;

.....

.....

.....

Label:

Statement;

.....

.....

Here label is any valid C identifier and is followed by colon.

The goto statement transfers the control to the statement after the label.

If the label is placed after the goto statement then the control is transferred forward and it is known as forward jump. If the label is placed before the goto statement then the control is transferred backward and the jump is called backward jump.

In the forward jump all the statements between the goto statement and the label are skipped .

In case of backward jump all the statement between the goto statement and the label are executed.

.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -04 (UNIT-03)**ITERATIONS/LOOPS**

Iterations or loops are used when we want to execute a statement or block of statements several times.

The repetition of loops is controlled with the help of a test condition. The statements in the loop keep on

executing repetitively until the test condition becomes false.

There are the following three types of loops:-

1. While loop
2. Do-while loop
3. For loop

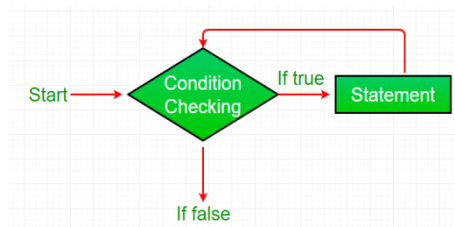
All these loops are explained as under:-

While loop : - A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.

Syntax :

```
while (boolean condition)
{
    loop statements...
}
```

Flowchart:



```
#include <stdio.h>
```

```
int main()
{
```

```
    int i = 5;
```

```
    while (i < 10) {
        printf("GFG\n");
```

```
    i++;  
}  
  
return 0;  
}
```

Output:

GFG
GFG
GFG
GFG
GFG

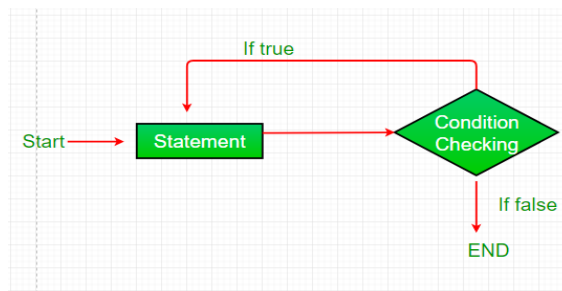
do-while loop:

do while loop is similar to while loop with the only difference that it checks for the condition after executing the statements, and therefore is an example of Exit Control Loop.

Syntax:

```
do  
{  
    statements..  
}  
while (condition);
```

Flowchart:



Example:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int i = 5;
```

```
    do {
```

```
    printf("GFG\n");
    i++;
} while (i < 10);

return 0;
}
```

Output:

GFG
GFG
GFG
GFG
GFG

Here is the difference table:

while	do-while
Condition is checked first then statement(s) is executed.	Statement(s) is executed atleast once, thereafter condition is checked.
It might occur statement(s) is executed zero times, If condition is false.	At least once the statement(s) is executed.
No semicolon at the end of while.	Semicolon at the end of while.
while(condition)	while(condition);
If there is a single statement, brackets are not required.	Brackets are always required.
Variable in condition is initialized before the execution of loop.	variable may be initialized before or within the loop.
while loop is entry controlled loop.	do-while loop is exit controlled loop.
while(condition)	do { statement(s); }
{ statement(s); }	while(condition);

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -05 (UNIT-03)

For loop :- The general syntax of for loop consists of three expressions separated by semicolons.

It is

given as follows:-

for(expression1;expression2;expression3)

Body of loop

condition

Next statement

out of loop

{

Statement(s);

}

Here the expression1 is the initialization expression, expression2 is the test expression and expression3 is the update expression.

Expression1 is executed only once when the loop starts and is used to initialize the loop variables.

Expression2 is a condition and is tested before each iteration of the loop.

Expression3 is an update expression and is executed each time after the body of the loop is executed.

The flow chart of for loop is given as follows:-

false

true

we give the following program to explain for loop

/* program to generate the Fibonacci series*/

#include<stdio.h>

#include<conio.h>

void main()

{

int x,y,z;

int i,n;

x=0;

y=1;

printf("enter the number of terms");

scanf("%d",&n);

printf("%d",y);

for(i=1;i<n;i++)

{

z=x+y;

printf("%d",z);

x=y;

Initialization

condition

Body of loop

Update

expression

Next statement after the

loop

```
y=z;
```

```
}
```

```
printf("\n");
```

```
getch();
```

```
}
```

All the three expressions in the for loop are optional and therefore we can omit any or all the three

expressions but in any case the two separating semi colons should always be present.

If we omit the test expression then it is always assumed to be true and therefore the loop never terminates and becomes an infinite loop.

If we want to make such loops a finite loop then a separate terminate condition is given within the loop

body

Jump statements

Jump statements are used to transfer the control from one part of the program to another part.

The various jump statements in C are as under:-

1. break statement

2. continue statement

3. goto statement

break statement:- break statement is used inside the loops or switch statement. This statement causes

an immediate exit from the loop or the switch case block in which it appears.

It can be written as

```
break;
```

Beginning of loop

True

Next Iteration

Of loop

False

Loop statements before

```
break
```

condition

Loop statements after

```
break
```

Break statement

Next statement

Out of loop

```
/*Program to understand the use of break statement*/
```

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int n;
```

```
for(n=1;n<=5;n++)
```

```
{
```

```
if(n==3)
```

```
{
```

```
printf("I understand the use of break \n");
```

```
break;
```

```
}
```

```
printf("Number = %d \n",n);
```

```
}
```

```
printf("Out of for loop \n");
```

```
}
```

Output:

Number = 1

Number = 2

I understand the use of break

Out of for loop

when break statement is encountered, loop is terminated and the control is transferred to the statement immediately after the loop.

If break statement is written inside a nested loop structure then it causes exit from the innermost loop.

In Switch case , it is used as the last statement of every case except the last one. When executed, it

transfers the control out of switch case and the execution of the program continues from the statement

following switch statement.

```
Switch(expression)
```

```
{
```

```
case1: statement-1;
```

```
break;
```

```
case2: statement-2;
```

```
break;
```

```
case3: statement-3;
```

```
break;
```

```
.
```

```
.
```

```
.
```

```
.
```

```
case n: statement-n;
```

```
break;
default: statement-d;
}
/*program illustrating use of break in switch case */
The Output of this program would be :
I am in case 2
```

continue statement :- The continue statement is used inside the body of the loop statement when we want to go to the next iteration of the loop after skipping some of the statements of the loop.

The continue statement is written as under:

```
continue;
```

Next iteration of loop True

The Output of this program would be :

The continue statement is used inside the body of the loop statement

when we want to go to the next iteration of the loop after skipping some of the statements of the loop.

The continue statement is written as under:-

Beginning of Loop

Next iteration of loop True

False

Loop statements before continue

Condition

Loop statements after continue

Continue

statement

The continue statement is used inside the body of the loop statement. It is used

when we want to go to the next iteration of the loop after skipping some of the statements of the loop.

Continue

statement

It is generally used with a condition. When a continue statement is encountered all the remaining

statements (statements after continue) in the current iteration are not executed and the loop continues

with the next iteration.

The difference between break and continue is that when a break statement is encountered the loop

terminates and the control is transferred to the next statement following the loop, but when a continue

statement is encountered the loop is not terminated and the control is transferred to the beginning of the

loop.

In while and do-while loops, after continue statement the control is transferred to the test condition and then the loop continues, whereas in for loop after continue statement the control is transferred to update expression and then the condition is tested.

Output :

Number = 1

Number =2

I understand the use of continue

Number =4

Number = 5

Out of for loop

.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

